

Summary

- Summary of Methodology
 - Data Collection: Accessed SpaceX launch data via API
 - Data Cleaning and Preparation: Cleaned and formatted data, conducted exploratory data analysis
 - Feature Engineering: Created new features and standardized data
 - Interactive Visualizations: Mapped launch sites and success rates, built interactive dashboard
 - Modeling Building and Evaluation: Implemented SVM, Trees, K-Nearest Neighbor
- Summary of Results
 - Data Insights: Identified factors influencing Falcon 9
 - Model Performance: SVM and K-Nearest Neighbor was 83.33% accuracy
 - Decision Tree: 94% accuracy
 - Key Findings: Launch site and payload mass impact landing success
 - Decision tree model is most effective

Introduction

- Project Background
 - Predicting successful landings of Falcon 9 first stage
 - Accurately predicting launching success will allow us to estimate cost and provide valuable insight for companies bidding against SpaceX
- Problems We Want to Find Answers to:
 - What factors influence successful landing?
 - How can we accurately predict landing outcomes?
 - Which machine learning models performs best?

Data Collection

Step 1: SpaceX API Request



Step 2: Web Scraping Wikipedia



Step 3: Data Integration



Step 1: API Request

- Use Python request library to connect to SpaceX API

Step 2: Parse API Response

- Convert API from JSON to Python Library

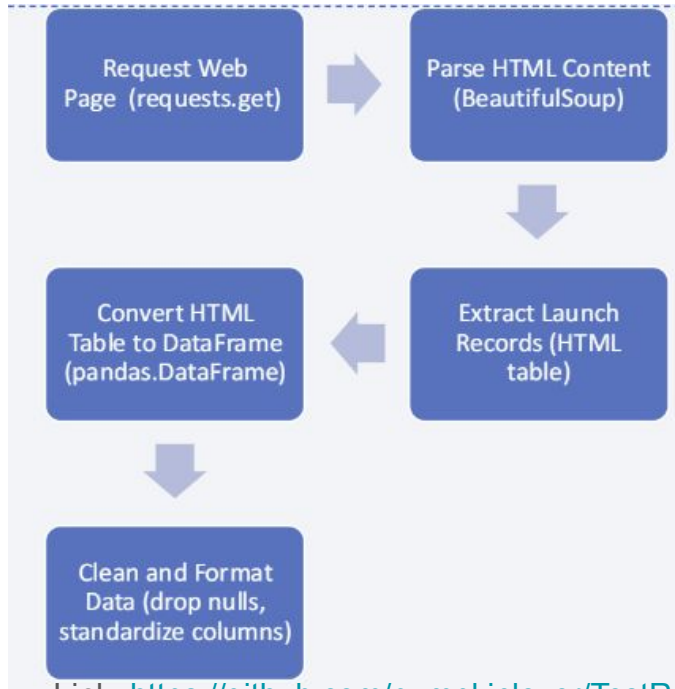
Step 3: Store Data Locally

- Save extracted data into Pandas Dataframe

Link:

[https://github.com/pumpkinlover/TestRepo/blob/main/jupyter-labs-spacex-data-collection-api%20\(1\).ipynb](https://github.com/pumpkinlover/TestRepo/blob/main/jupyter-labs-spacex-data-collection-api%20(1).ipynb)

Web Scrapping



Step 1: Initiate Web Scrapping

- Use python request library to fetch content

Step 2: Parse HTML Content

- Use BeautifulSoup to parse HTML content
- Extract HTML table

Step 3: Convert to DataFrame

- Convert extracted data into panda dataframes
- Clean and format data

Link: <https://github.com/pumpkinlover/TestRepo/blob/main/jupyter-labs-webscraping.ipynb>

Data Wrangling

Step 1: Data Cleaning

- Identify and remove any missing values

Step 2: Data Transformation

- Convert data into appropriate form
- Scale features from existing data
- Standardize text

Link:

<https://github.com/pumpkinlover/TestRepo/blob/main/labs-jupyter-spacex-Data%20wrangling.ipynb>

Step 3: Data Integration

- Merge datasets
- Ensure consistent column names

Step 4: Data Validation

- Check for duplicate records
- Verify accuracy and consistency

EDA with Data Visualization

Charts Plotted

1. **Histogram**

- Purpose: Visualize distribution of numerical values

2. **Bar Chart**

- Purpose: Used to compare categorical variables

3. **Line Chart**

- Used to track trends overtime

4. **Scatter Plot**

- Purpose: Used to explore relationships between two variables

5. **Heatmap**

- Purpose: Used to visualize correlation matrices between numerous numerical variables

6. **Box Plot**

- Purpose: Used to visualize distribution of numerical data

Link: <https://github.com/pumpkinlover/TestRepo/blob/main/edadataviz.ipynb>

EDA with SQL

Aggregate Queries

- Calculated total number of launches
- Counted successful and failed launches
- Calculated success rates

Join Queries

- Joined tables

Filtering Queries

- Filtered data to focus on specific launch outcomes

Link:

[https://github.com/pumpkinlover/TestRepo/blob/main/jupyter-labs-eda-sql-coursera_sqlite%20\(1\).ipynb?short_path=aa861c8](https://github.com/pumpkinlover/TestRepo/blob/main/jupyter-labs-eda-sql-coursera_sqlite%20(1).ipynb?short_path=aa861c8)

Sorting Queries

- Sorted data to identify trends and outliers

Subqueries

- Nested queries to calculate derived metrics
- Used to perform detailed analysis

Interactive Visual Analytics with Folium Map

Markers

- Placed to indicated launch sites
- Helps users identify specific SpaceX launch site locations

Circles

- Added to visualize proximity zones
- Illustrate potential impact zones
- Visualize safety parameters

Interactive Visual Analytics

Lines

- Draw lines to connect launch sites with other relevant sites
- Shows connection between launch sites and other locations

Link:

https://github.com/pumpkinlover/TestRepo/blob/main/lab_jupyter_launch_site_location.ipynb

Plotly Dash

Plots/Graphs Added

- Success Pie Chart
- Displays distribution of failed and successful launches
- Helps understand overall performance metrics
- Success Payload Scatter Plot
- Shows relationship between payload mass and launch success
- Helps identify correlations between payload characteristics and launch outcomes

Interactions Added

- Launch Site Dropdown
- Allows users to select specific launch sites
- Allows launch site comparisons between different regions
- Ranger Slider for Payload
- Allows users to adjust payload mass ranges
- Enables detailed analysis into payload performance related factors

Predictive Analysis

Data Processing

- Standardized features to ensure all variables are contributing equally

Model Selection

- Explored multiple classification algorithms

Hyperparameter Tuning

- Used GridSearchCV to search for optimal hyperparameters

Model Evaluation

- Used cross-validation techniques
- Utilized various metrics to assess performance

Improvement

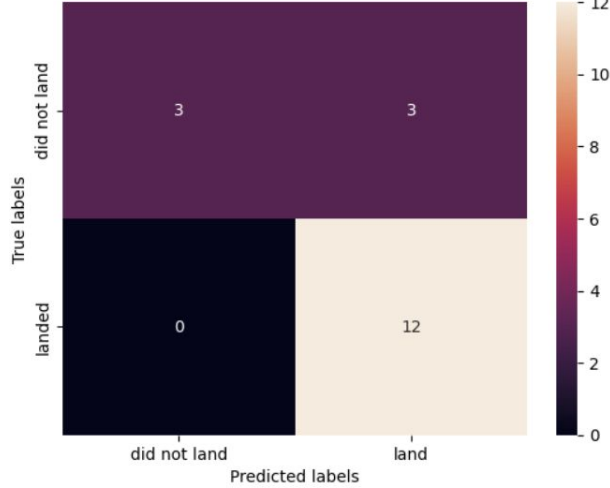
- Adjusted models based on insight and maximize predictability and accuracy

Selection of Best Performing Model

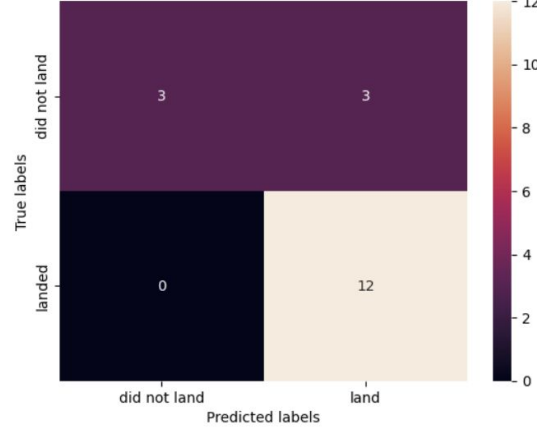
- Identified model with highest accuracy

Predictive Analysis

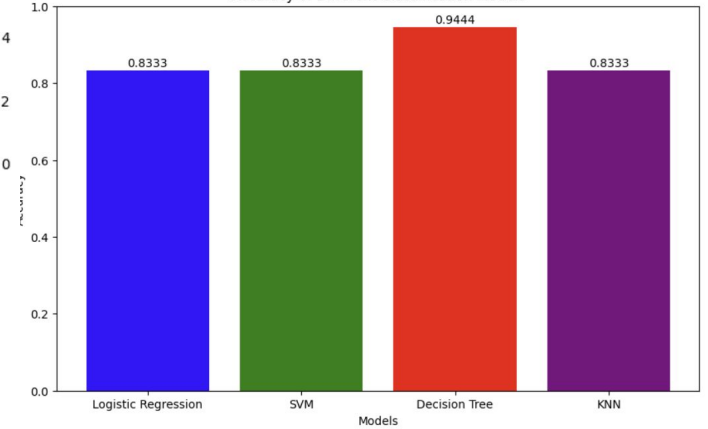
Confusion Matrix



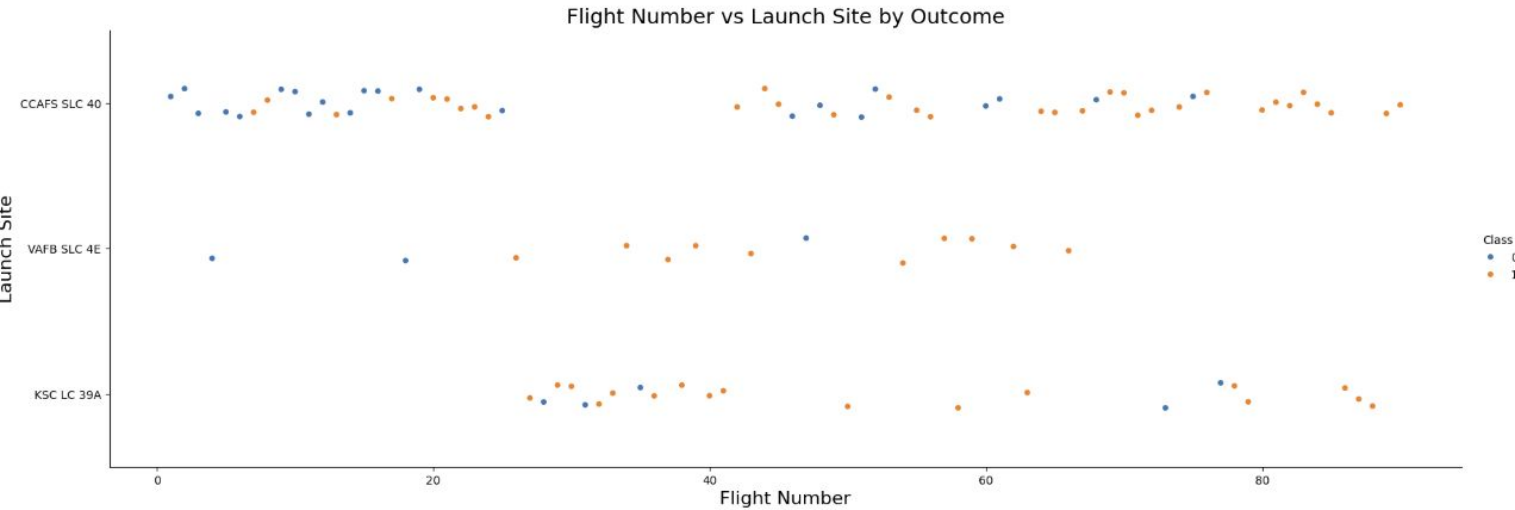
Confusion Matrix



Accuracy of Different Classification Models

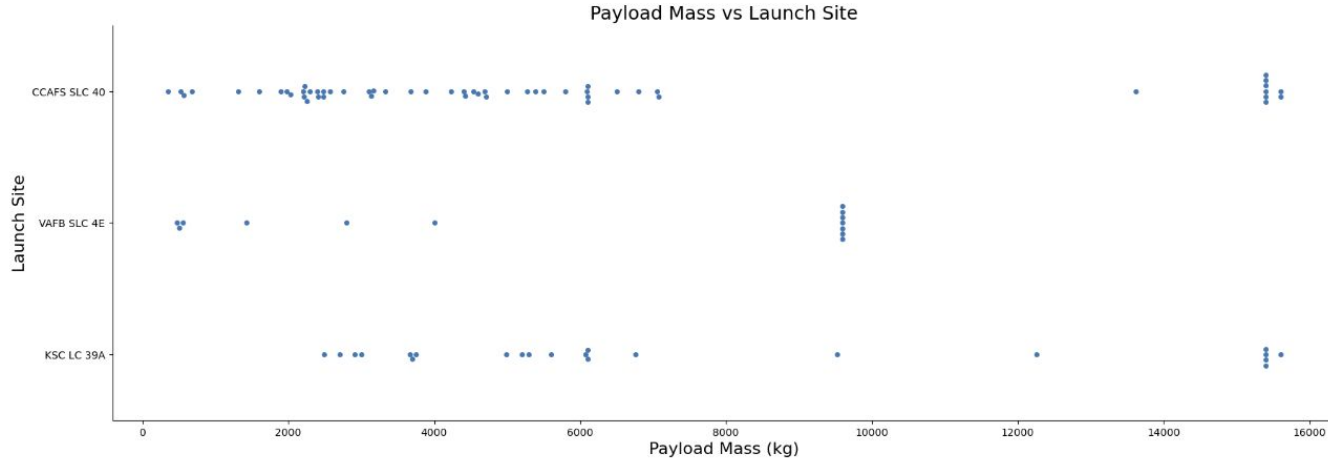


EDA Visualizations: Flight Number vs Launch Site



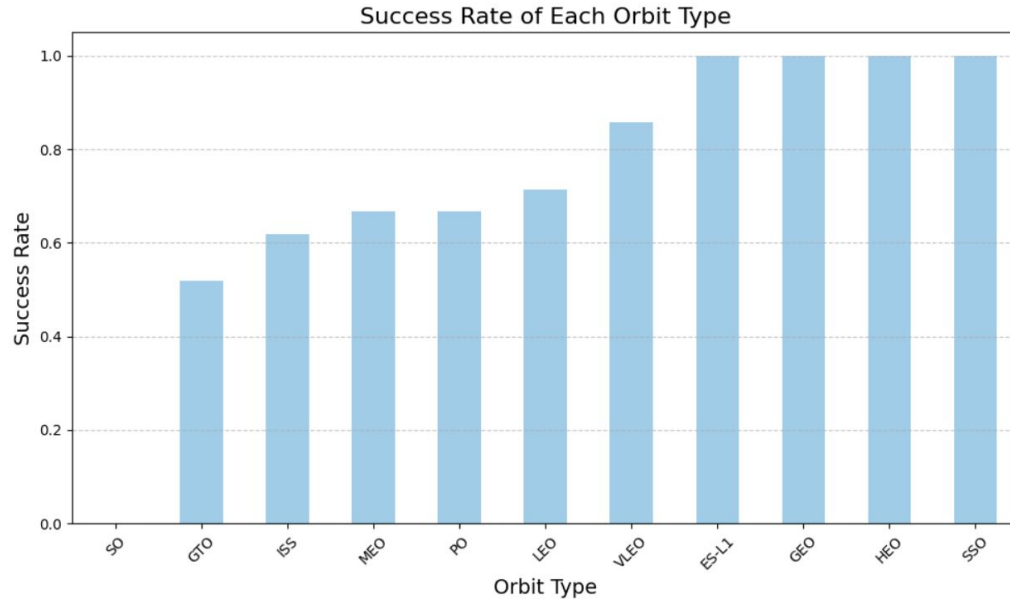
- CCAFS and KSC LC have a mix of successful and unsuccessful landings, indicating other factors might contribute to landing success

EDA Visualization: Payload vs Launch Site



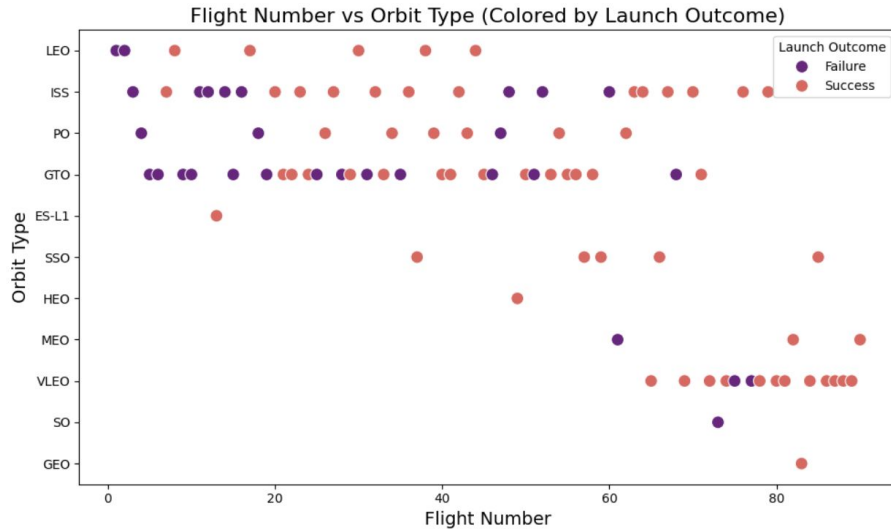
- Most launches from CCAFS had payloads below 10,000 kg, while VAFB and KSC LC have a wider range of payload masses, which indicates different mission profiles

EDA Visualizations: Success vs Orbit Type



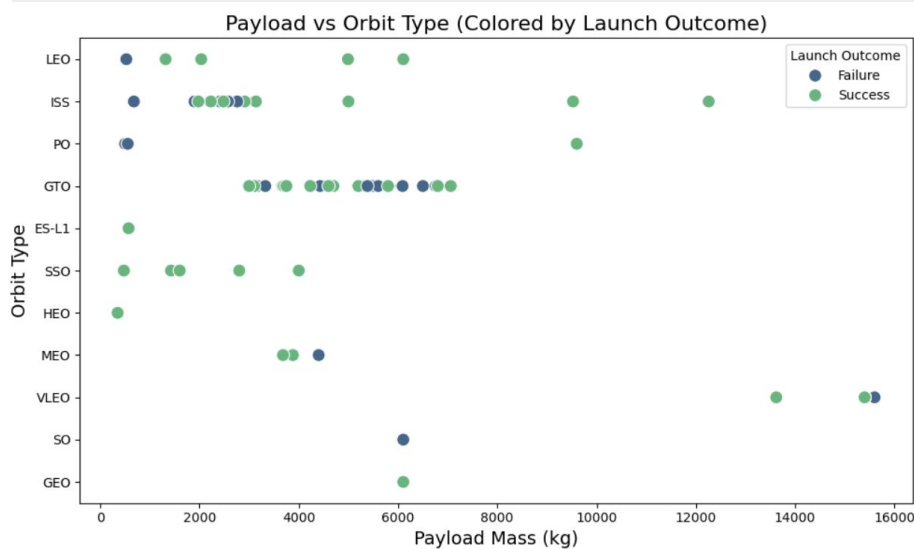
- VLEO, ES-L1, GEO, HEO, and SSO had perfect success rate, indicating these orbits are highly reliable

EDA Visualization: Flight Number vs Orbit Type



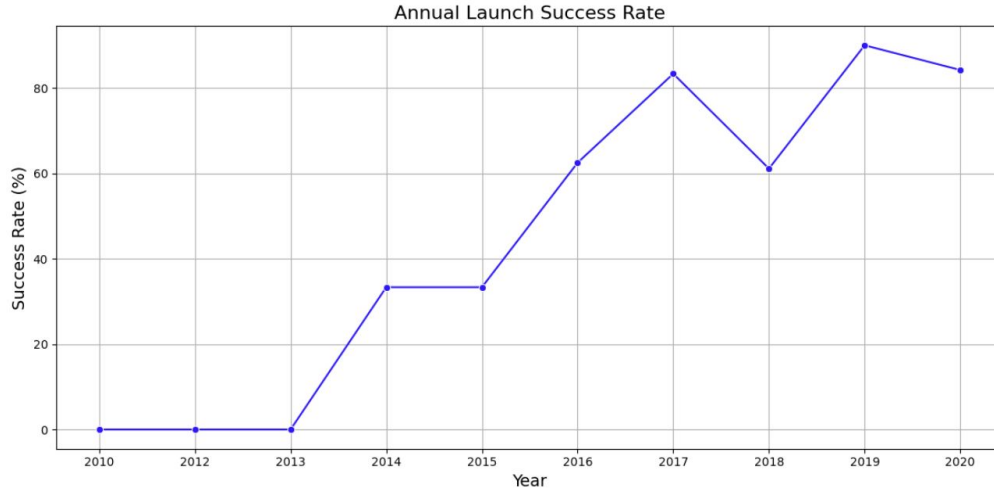
- Success rate improved overtime with higher flight numbers

EDA Visualization: Payload vs Orbit Type



- Successful landings are frequent across all orbit types
- Higher payload masses show more mixed success rates

EDA Visualization: Yearly Success Rate



- Annual success rate has improved tremendously after 2013

EDA with SQL Results: Launch Site Names

```
: %sql SELECT DISTINCT "Launch_Site" FROM SPACEXTABLE;
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
: Launch_Site
```

```
CCAFS LC-40
```

```
VAFB SLC-4E
```

```
KSC LC-39A
```

```
CCAFS SLC-40
```

EDA with SQL Results: Launch Sites with CCA

```
%sql SELECT * FROM SPACEXTABLE WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5;
```

```
* sqlite:///my_data1.db
```

```
Done.
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

EDA with SQL Results: Payload

Task 3

Display the total payload mass carried by boosters launched by NASA (CRS)

```
: %sql SELECT SUM("PAYLOAD_MASS_KG_") FROM SPACEXTABLE WHERE "Customer" = 'NASA (CRS)';
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
: SUM("PAYLOAD_MASS_KG_")
```

```
45596
```

Task 4

Display average payload mass carried by booster version F9 v1.1

```
: %sql SELECT AVG("PAYLOAD_MASS_KG_") FROM SPACEXTABLE WHERE "Booster_Version" = 'F9 v1.1';
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
: AVG("PAYLOAD_MASS_KG_")
```

```
2928.4
```

EDA with SQL Results: Successful Launch Date

List the date when the first succesful landing outcome in ground pad was acheived.

Hint: Use min function

```
: %sql SELECT MIN("Date") FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (ground pad)';
```

```
* sqlite:///my_data1.db
```

Done.

```
: MIN("Date")
```

2015-12-22

EDA with SQL Results: Boosters with Success and Number of Successful and Failed Outcomes

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

```
%sql SELECT DISTINCT "Booster_Version" FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (drone ship)' AND "PAYLOAD_MASS_KG_" > 4000 AND "F
```

```
* sqlite:///my_data1.db  
Done.
```

Booster_Version

F9 FT B1022

F9 FT B1026

F9 FT B1021.2

F9 FT B1031.2

Task 7

List the total number of successful and failure mission outcomes

```
%sql SELECT "Mission_Outcome", COUNT(*) AS "Total" FROM SPACEXTABLE WHERE "Mission_Outcome" IN ('Success', 'Failure') GROUP BY "Mission_Outcome"
```

```
* sqlite:///my_data1.db  
Done.
```

Mission_Outcome	Total
-----------------	-------

Success	98
---------	----

EDA with SQL Results: Boosters with Maximum Payload Mass

List all the booster_versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

```
: %sql SELECT DISTINCT "Booster_Version" FROM SPACEXTABLE WHERE "PAYLOAD_MASS_KG_" = (SELECT MAX("PAYLOAD_MASS_KG_") FROM SPACEXTABLE);  
* sqlite:///my_data1.db  
Done.
```

```
: Booster_Version
```

F9 B5 B1048.4

F9 B5 B1049.4

F9 B5 B1051.3

F9 B5 B1056.4

F9 B5 B1048.5

F9 B5 B1051.4

F9 B5 B1049.5

F9 B5 B1060.2

F9 B5 B1058.3

F9 B5 B1051.6

F9 B5 B1060.3

F9 B5 B1049.7

EDA with SQL Results: Failed Landing Outcomes

Note: SQLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

```
1: %%sql
SELECT
CASE
    WHEN substr("Date", 6, 2) = '01' THEN 'January'
    WHEN substr("Date", 6, 2) = '02' THEN 'February'
    WHEN substr("Date", 6, 2) = '03' THEN 'March'
    WHEN substr("Date", 6, 2) = '04' THEN 'April'
    WHEN substr("Date", 6, 2) = '05' THEN 'May'
    WHEN substr("Date", 6, 2) = '06' THEN 'June'
    WHEN substr("Date", 6, 2) = '07' THEN 'July'
    WHEN substr("Date", 6, 2) = '08' THEN 'August'
    WHEN substr("Date", 6, 2) = '09' THEN 'September'
    WHEN substr("Date", 6, 2) = '10' THEN 'October'
    WHEN substr("Date", 6, 2) = '11' THEN 'November'
    WHEN substr("Date", 6, 2) = '12' THEN 'December'
    ELSE 'Unknown'
END AS "Month_Name",
"Mission_Outcome",
"Booster_Version",
"Launch_Site"
FROM
SPACEXTABLE
WHERE
substr("Date", 0, 5) = '2015';
```

```
* sqlite:///my_data1.db
Done
```

Month_Name	Mission_Outcome	Booster_Version	Launch_Site
January	Success	F9 v1.1 B1012	CCAFS LC-40
February	Success	F9 v1.1 B1013	CCAFS LC-40
March	Success	F9 v1.1 B1014	CCAFS LC-40
April	Success	F9 v1.1 B1015	CCAFS LC-40
April	Success	F9 v1.1 B1016	CCAFS LC-40
June	Failure (in flight)	F9 v1.1 B1018	CCAFS LC-40
December	Success	F9 FT B1019	CCAFS LC-40

EDA with SQL Results: Ranking Count of Landing Outcomes

```
%%sql

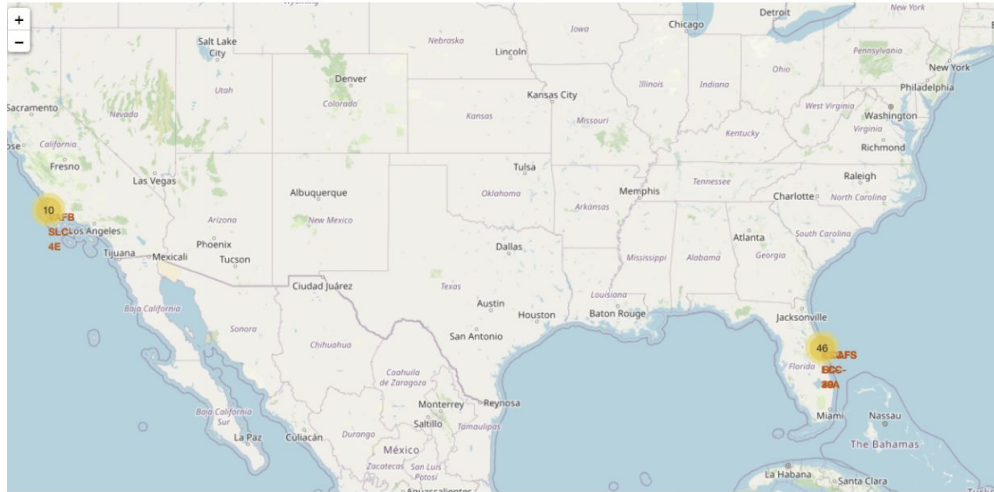
SELECT
    "Landing_Outcome",
    COUNT(*) AS "Count"
FROM
    SPACEXTABLE
WHERE
    "Date" BETWEEN '2010-06-04' AND '2017-03-20'
GROUP BY
    "Landing_Outcome"
ORDER BY
    COUNT(*) DESC;
```

```
* sqlite:///my_data1.db
Done.
```

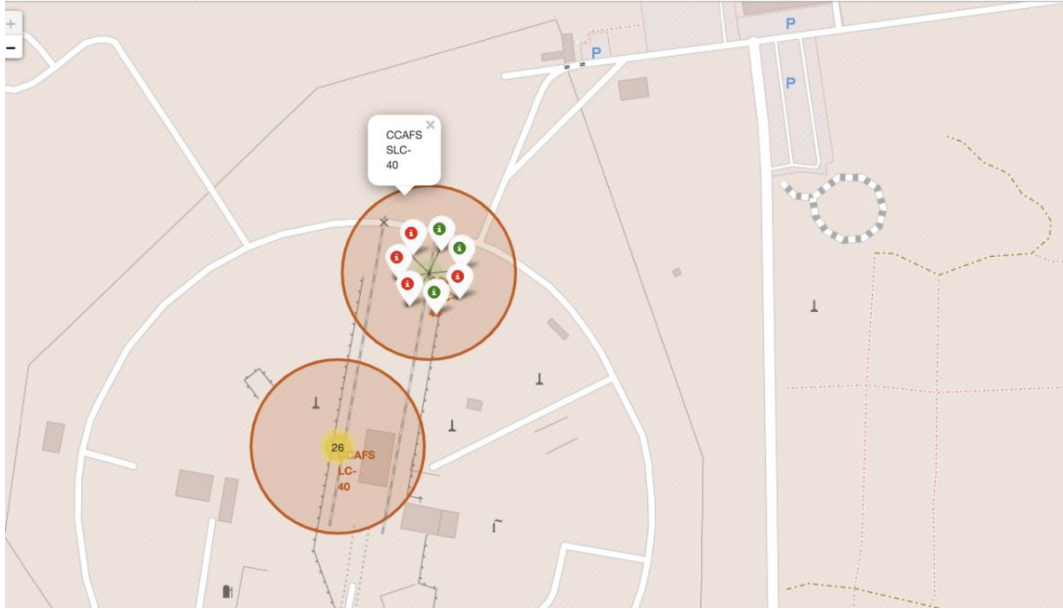
Landing_Outcome	Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

Launch Sites and Proximity Analysis

- Not all launch sites are close to the equator, but are close to the coast

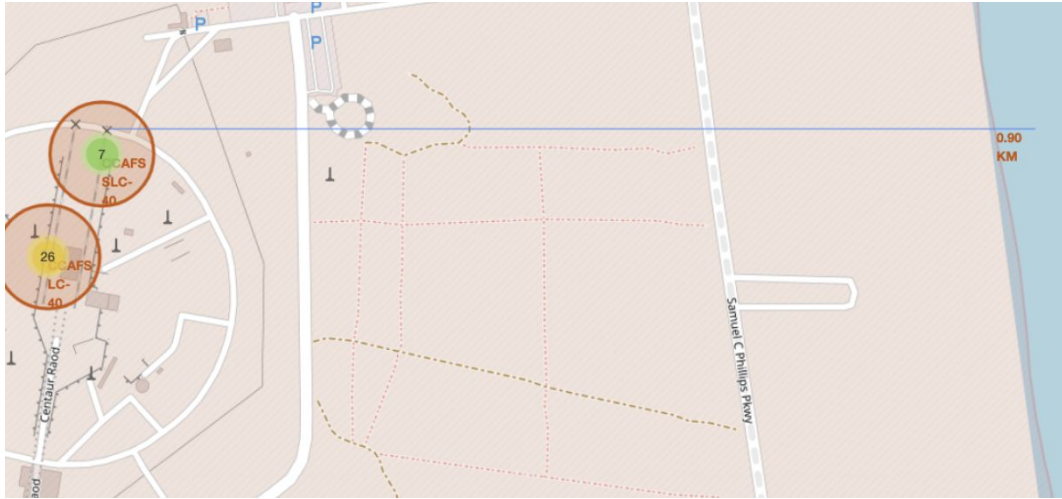


Launch Sites and Proximity Analysis



- Enhanced visualization allows for better exploration and analysis for launch data

Launch Sites and Proximity Analysis



- Provides a visual representation of distance between CCAFS and the closest coast line

